

Data Structure

第一章 绪论

1. 逻辑结构（抽象） -- 一对多 -- 存储结构（具体）
2. 计算 π 的算法有零个输入
3. 主定理: $T(n) = aT(n/b) + f(n)$
 - 若 $f(n) \neq n^{\log_b a}$, 则 $T(n) = \Theta(\max(f(n), n^{\log_b a}))$
 - 否则, $T(n) = \Theta(n^{\log_b a} \log n)$
 - **不等取大, 相等乘对数因子**

第二章 线性表

1. 线性表中: 数据元素 -- 一对多 -- 数据项
 - 数据元素就是线性表里存的元素
 - 数据元素是由数据项组成的
2. 单链表逆置方法: 摘下头结点, 从首元开始, 一次插入到头结点后面, 直到最后一个节点为止
3. 取链表中点方法: 设置快慢指针分别为 f 和 s, 初始时均指向首元, 之后 s 每次走一步, f 每次走两步。当 f 指向表尾时, s 正好指向链表中点
4. 有时候, 可以将单链表的变成循环单链表来加速一些循环操作的问题, 比如单链表循环位移

第三章 栈、队列和数组

1. 当 n 个不同元素入栈时, 出栈元素不同排列的个数为 $\frac{1}{n+1}C_{2n}^n$
 - 组合计算公式: $C_{2n}^n = \frac{A_{2n}^n}{n!}$
2. 一个存储单元 = 1 byte, 地址 1000H + 1byte = 1001H
 - 1000H + 1KB = 1400H
3. C 语言标识符只能以英文字母或下划线开头, 不能以数字开头
4. a b c d e 依次入栈, 在所有可能的出栈序列中, 以 d 开头的序列怎么求
 - d 之后入栈的只有 e
 - d 最先出栈, 下面的只能按照 c b a 的顺序出栈
 - e 有可能插入到 c b a 之间的任意位置出栈
 - 故
 - d e c b a
 - d c e b a
 - d c b e a
 - d c b a e
5. 循环队列首尾指针
 - 不管是队首指针指向首元的前驱还是队尾指针指向尾元的后继以牺牲一个单元来区分对空和队满, 以下结论均成立
 - 判断队满: $(rear + 1) \% SIZE == front$
 - 判断队空: $front == rear$ // TODO 似乎存在错误
 - 求元素个数: $(rear - front + SIZE) \% SIZE$
 - 队尾指针指向首元的前驱, 队首指针指向尾元: 队尾指针需要初始化在数组末尾, 即队尾指针 = $n - 1$
 - 队首指针指向首元的前驱, 队尾指针指向尾元: 队首指针需要初始化在数组末尾, 即队首指针 = $n - 1$

◦ 队首指针的取值范围是适合做链式队列

6. 带头指针的单链表最适合做链式队列

◦ 尾删 $O(n)$, 尾插、头插、头删 $O(1)$

7. 带头尾指针的单链表适合做链式队列

◦ 尾删 $O(n)$, 尾插、头插、头删 $O(1)$

8.

名称	指针名	作用
队头	front	弹出元素
队尾	rear	插入元素

◦ 头删尾插

9. 使用带头尾指针的单链表做链式队列时，删除操作可能头尾指针都要修改。因为当队列只有一个元素时，需要修改尾指针

10. 三元组表存储稀疏矩阵无法随机访问

11. 三对角矩阵公式推导

```
1  第 i - 1 行: 2 + (i - 2) * 3 // 2 是第一行的两个, 后面都是 3 个一行
2  第 i 行      : j - i + 2
3  故 k = 2 + (i - 2) * 3 + j - i + 2 - 1 = 2i + j - 3 // 下标从 0 开始
4      k = 2 + (i - 2) * 3 + j - i + 2 = 2i + j - 2 // 下标从 1 开始
5  对于下标从 0 开始的情况
6  i = (k + 1) / 3 + 1
7  j = k - 2i + 3
```

12. 矩阵与压缩数组下标换算公式

◦ 对称矩阵

矩阵起点	数组起点	矩阵 → 数组公式	数组 → 矩阵公式
0	0	$k = \begin{cases} \frac{i(i+1)}{2} + j & i \geq j \\ \frac{j(j+1)}{2} + i & j > i \end{cases}$	$i = \left\lfloor \frac{\sqrt{8k+1}-1}{2} \right\rfloor, j = k - \frac{i(i+1)}{2}$
0	1	$k = \begin{cases} \frac{i(i+1)}{2} + j + 1 & i \geq j \\ \frac{j(j+1)}{2} + i + 1 & j > i \end{cases}$	$i = \left\lfloor \frac{\sqrt{8(k-1)+1}-1}{2} \right\rfloor, j = (k-1) - \frac{i(i+1)}{2}$
1	0	$k = \begin{cases} \frac{(i-1)i}{2} + j & i \geq j \\ \frac{(j-1)j}{2} + i & j > i \end{cases}$	$i = \left\lfloor \frac{\sqrt{8k+1}-1}{2} \right\rfloor + 1, j = k - \frac{(i-1)i}{2}$
1	1	$k = \begin{cases} \frac{(i-1)i}{2} + j + 1 & i \geq j \\ \frac{(j-1)j}{2} + i + 1 & j > i \end{cases}$	$i = \left\lfloor \frac{\sqrt{8(k-1)+1}-1}{2} \right\rfloor + 1, j = (k-1) - \frac{(i-1)i}{2}$

◦ 下三角矩阵

矩阵起点	数组起点	矩阵 → 数组公式	数组 → 矩阵公式
0	0	$k = \frac{i(i+1)}{2} + j$ (当且仅当 $i \geq j$)	$i = \left\lfloor \frac{\sqrt{8k+1}-1}{2} \right\rfloor, j = k - \frac{i(i+1)}{2}$
0	1	$k = \frac{i(i+1)}{2} + j + 1$ (当且仅当 $i \geq j$)	$i = \left\lfloor \frac{\sqrt{8(k-1)+1}-1}{2} \right\rfloor, j = (k-1) - \frac{i(i+1)}{2}$
1	0	$k = \frac{(i-1)i}{2} + j - 1$ (当且仅当 $i \geq j$)	$i = \left\lfloor \frac{\sqrt{8k+1}-1}{2} \right\rfloor + 1, j = k - \frac{(i-1)i}{2} + 1$

矩阵起点	数组起点	矩阵 → 数组公式	数组 → 矩阵公式
1	1	$k = \frac{(i-1)i}{2} + j$ (当且仅当 $i \geq j$)	$i = \left\lfloor \frac{\sqrt{8(k-1)+1}-1}{2} \right\rfloor + 1,$ $j = (k-1) - \frac{(i-1)i}{2} + 1$

◦ 上三角矩阵

矩阵起点	数组起点	矩阵 → 数组公式	数组 → 矩阵公式
0	0	$k = \frac{n(n+1)}{2} - \frac{(n-i)(n-i+1)}{2} + j - i$ (当且仅当 $j \geq i$)	$i = n - \left\lfloor \frac{\sqrt{8(\frac{n(n+1)}{2}-k)+1}-1}{2} \right\rfloor,$ $j = k + i - \frac{n(n+1)}{2} + \frac{(n-i)(n-i+1)}{2}$
0	1	$k = \frac{n(n+1)}{2} - \frac{(n-i)(n-i+1)}{2} + j - i + 1$ (当且仅当 $j \geq i$)	$i = n - \left\lfloor \frac{\sqrt{8(\frac{n(n+1)}{2}-(k-1))+1}-1}{2} \right\rfloor,$ $j = (k-1) + i - \frac{n(n+1)}{2} + \frac{(n-i)(n-i+1)}{2}$
1	0	$k = \frac{(n-1)n}{2} - \frac{(n-i)(n-i+1)}{2} + j - i$ (当且仅当 $j \geq i$)	$i = n - \left\lfloor \frac{\sqrt{8(\frac{(n-1)n}{2}-k)+1}-1}{2} \right\rfloor + 1,$ $j = k + i - \frac{(n-1)n}{2} + \frac{(n-i)(n-i+1)}{2}$
1	1	$k = \frac{(n-1)n}{2} - \frac{(n-i)(n-i+1)}{2} + j - i + 1$ (当且仅当 $j \geq i$)	$i = n - \left\lfloor \frac{\sqrt{8(\frac{(n-1)n}{2}-(k-1))+1}-1}{2} \right\rfloor + 1,$ $j = (k-1) + i - \frac{(n-1)n}{2} + \frac{(n-i)(n-i+1)}{2}$

◦ 三对角矩阵

矩阵起点	数组起点	矩阵 → 数组公式	数组 → 矩阵公式
0	0	$k = 3i + j - i$ (当且仅当 $j = i - 1, i, i + 1$)	$i = \left\lfloor \frac{k}{3} \right\rfloor, j = k \% 3 + i - 1$
0	1	$k = 3i + j - i + 1$ (当且仅当 $j = i - 1, i, i + 1$)	$i = \left\lfloor \frac{k-1}{3} \right\rfloor,$ $j = (k-1) \% 3 + i - 1$
1	0	$k = 3(i-1) + j - (i-1)$ (当且仅当 $j = i - 1, i, i + 1$)	$i = \left\lfloor \frac{k}{3} \right\rfloor + 1,$ $j = k \% 3 + (i-1) - 1$
1	1	$k = 3(i-1) + j - (i-1) + 1$ (当且仅当 $j = i - 1, i, i + 1$)	$i = \left\lfloor \frac{k-1}{3} \right\rfloor + 1,$ $j = (k-1) \% 3 + (i-1) - 1$

◦ 行优先与列优先的区别为 **i** 与 **j** 互换

13. 二维矩阵 $A[i][j]$ 与 $A[i-k][i-k]$ 之间的内存距离为常数 $d = k * (m+1) * L$

◦ k 为行和列的偏移量, m 为矩阵列数, L 为每个元素占用的内存大小

◦ 下标从 0 开始、下标从 1 开始、矩阵行列不等时该公式均试用

14. 下标从 0 开始时, 下标值表示该元素前面的元素个数; 下标从 1 开始时, 下标值减 1 表示该元素前面的元素个数。

第四章 串

1. KMP 求 next 数组

- next 数组表示模式串中最长相等前后缀的长度（对于一对相等的前后缀，不一定是回文，前后缀顺序相等）
- 若模式串下标从 0 开始：next 数组首元为 0，数最长相等前后缀的长度即可
 - 注意，这种只在代码里写
- 若模式串下标从 1 开始：先求下标从 0 开始的 next 数组，再将 next 数组整体加 1，整体右移 1 位，首元补 0
 - 此时模式串的下标为 1 和 2 的元素一定分别为 0 和 1
 - 这种情况考研常考
 - 考题中若模式串下标从 0 开始，先按代码中的方法求下标从 0 开始的 next 数组，然后整体右移 1 位，首元补 -1
 - 考题中若看到 next 数组首元为 -1，说明题设下标从 0 开始；若首元为 0，说明题设下标从 1 开始

2. KMP 求比较次数

- 失配了的那次比较也算

3. KMP 模式串最大滑动距离

- 不给主串，一律认为模式串匹配到最后一个字符时失配并且该字符在模式串中不存在，直接跳到模式串开头
- 最大距离 = 模式串长度 - 1

第五章 树与二叉树

树

- n : 树的节点数
- n_i : 度为 i 的节点数
- n_0 : 为 0 的节点个数/叶节点个数
- m : 树的度
- h : 树的高度

$$1. n = \sum_1^m i n_i + 1 = \sum_0^m n_i = n_0 + \sum_1^m n_i$$

$$2. \text{对于一颗满 } m \text{ 叉树, 第 } i \text{ 层有 } m^{i-1} \text{ 个节点, 到第 } i \text{ 层为止共有 } \frac{m^i - 1}{m - 1}$$

$$3. n_0 = \sum_2^m (i - 1) n_i + 1$$

$$4. h \text{ 的取值范围为 } \lceil \log_m n(m - 1) + 1 \rceil \leq h \leq n - m + 1$$

二叉树

- n : 二叉树节点数
- n_0 : 度为 0 的节点个数/叶节点个数
- n_2 : 度为 2 的节点个数/分支节点个数
- h : 树的高度

1. 非空二叉树:

$$n_0 = n_2 + 1, n = n_0 + n_1 + n_2$$

第 k 层最多有 2^{k-1} 个节点

整个树一共最多有 $2^h - 1$ 个节点

2. 完全二叉树:

$$h = \lfloor \log_2 n \rfloor + 1 = \lceil \log_2(n+1) \rceil$$

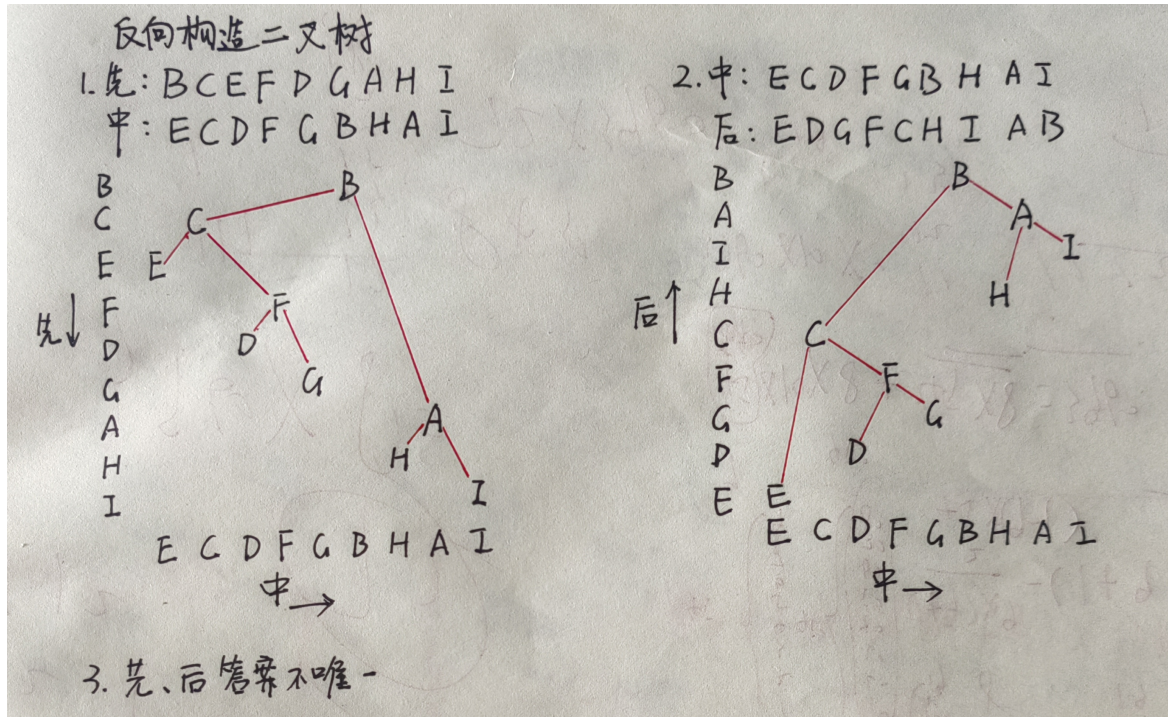
3. 有 n 个节点的二叉树采用链式存储，空指针数为 $n+1$

二叉树的遍历

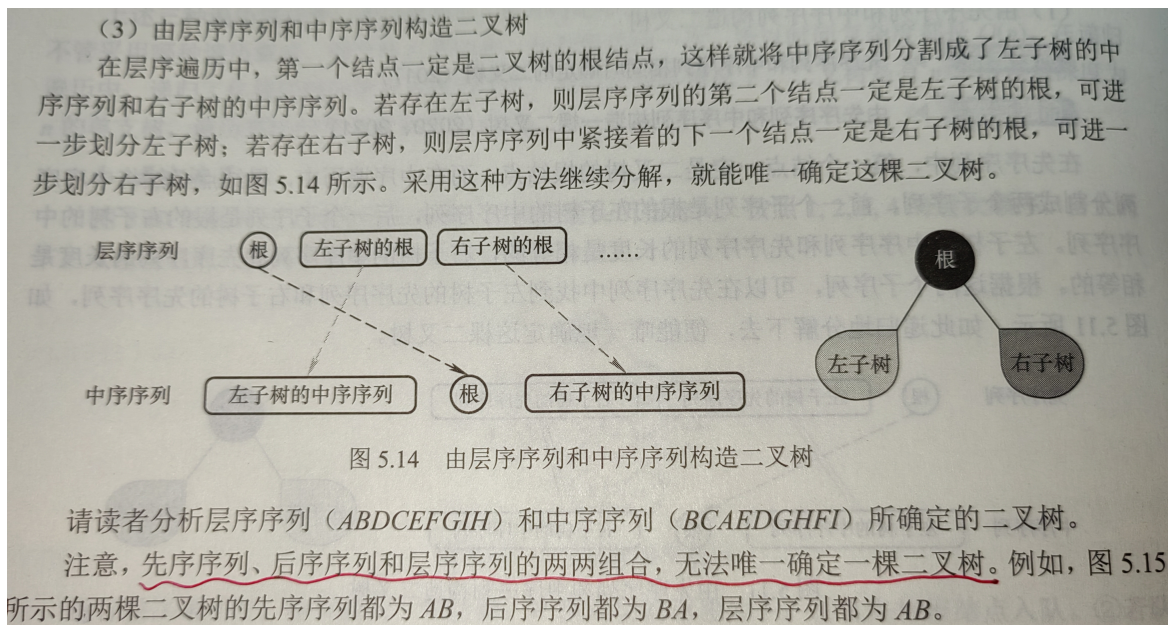
1. 已知先序、中序构造二叉树；已知后序、中序构造二叉树

注：

1. 连线的时候不能跨节点（对上面的节点做一条铅垂线，下面的连线不能跨过该铅垂线）
2. 中序写下面写上面无所谓
3. 构造完成后必须检查一遍，检验遍历顺序是否符合题目要求
4. 如果实在不能使用这个方法，那先序就从头看起，二分中序序列；后序从尾看起，二分中序序列



2. 已知层序、中序构造二叉树



3. 先序、后序和层序两两组合无法唯一确定一棵二叉树

4. 给定 先序 和 后序 序列，如何确定符合条件的二叉树数量

- 前序序列的首元素和后序序列的末元素是根节点，必须相同

- 遍历前序序列，对于每个节点 x （除最后一个），找到其后一个节点 y
 - 在后序序列中找到 x 的位置，其前一个节点若等于 y ，则说明 x 只有一个子节点（左或右），产生结构歧义
 - 每出现一次这种情况，可能的树数量翻倍
 - 故可能的二叉树数量为 2^k ，其中 k 是满足前序中某节点的后继等于后序中该节点的前驱的节点个数
 - eg：前序 ABC 后序 CBA，A 和 B 满足条件，个数为 $2^k = 2^2 = 4$ 种
 - 这个方法也可以用来确定二叉树的结构以及子节点
5. 二叉树是逻辑结构，线索二叉树是物理结构
 6. 二叉树线索化后，空链域的个数为 2
 7. 后序线索二叉树不能有效解决求后序后继的问题，故遍历后序线索二叉树需要栈的支持
 8. 给定长度为 n 的先序序列求可能的二叉树个数
 - 问题等同于给定长度为 n 的入栈序列，有多少种出栈序列
 - 个数为 $\frac{1}{n+1} C_{2n}^n$
 9. 非空二叉树先序序列和后序序列正好相反，二叉树的形态为：**高度等于节点数**
非空二叉树先序序列和后序序列正好相同，二叉树的形态为：**只有一个根节点**
 10. 结点的带权路径长度（WPL）：从树的根结点到任意结点的路径长度（经过的边数）与该结点上权值的乘积。
 - 实际上是哈夫曼树的知识

树、森林

1. 森林的先根遍历序列对应于其二叉树的先序遍历序列，森林的后根遍历序列对应于其二叉树的中序遍历序列

树与二叉树的应用

1. 哈夫曼树中只有度为 0 和 2 的节点，节点总数为 $n = n_0 + n_2$ ，且 $n_0 = n_2 + 1$
 - 哈夫曼树总码字数量为叶子节点个数，即 n_0
 - 显然 n 一定不为偶数，即哈夫曼树总节点数不能是偶数
 - 分支数 = 边数 = $n - 1$
 - 哈夫曼树中只有度为 0 和 m 的节点，分支数 = $m * n_m = n - 1 = n_0 + n_m - 1$

第六章 图

图的基本概念

1. 有向图中每条有向边都有一个起点和终点，可推出
 - 顶点入度之和 = 顶点出度之和 = 边数
2. 子图：可能少边，可能少点；生成子图：少边，不少点
3. 非连通图边最多的情况：由 $n - 1$ 个顶点构成一个完全图，此时再加入一个顶点则变成非连通图
4. 有向图强连通情况下边最少的情况：至少需要 n 条边，构成一条环路
5. 完全图（也称简单完全图）
 - 无向图：
 $|E|$ 的取值范围为 0 到 $\frac{n(n-1)}{2}$ 。有 $\frac{n(n-1)}{2}$ 条边的无向图称为 **完全图**，在完全图中任意两个顶点之间都存在边
 - 有向图：
 $|E|$ 的取值范围为 0 到 $n(n-1)$ 。有 $n(n-1)$ 条弧的有向图称为 **有向完全图**，在有向完全图中任意两个顶点之间都存在方向相反的两条弧
6. 稠密图、稀疏图

- 稀疏图：边数很少的图
- 稠密图：边数较多的图
- 稀疏和稠密是相对概念，一般当图 G 满足 $|E| < \sqrt{\log_2 |V|}$ 时，可视为稀疏图

7. 有向树

- 若一个 **有向图** 满足：
 - 存在且仅存在一个顶点入度为 0（称为根节点），
 - 其余所有顶点的入度均为 1
 - 则称该有向图为 **有向树**

8. 无向图有 n 个顶点， m 条边

- 顶点 V 的度： $TD(V)$
- $\sum_{i=1}^n TD(V_i) = 2 * m$

9. 插入边使得无向图连通分量最多：由于插入一条边，连通分量个数 -1，尽可能将边插入一个连通分量中，使之变成完全图

- 例如，具有 51 个顶点和 21 条边的无向图的连通分量最多为：7 个顶点的完全图有 21 条边，故连通分量个数为 $51 - 7 + 1 = 45$

10. 节点数为 n 的树，边数为 $n - 1$

11. n 个顶点的完全图，边数为 $\frac{n(n-1)}{2}$

- 要使 n 个顶点的无向图在任何情况下都是联通的，至少需要 $n - 1$ 个顶点构成完全图，再加一条边连接最后一个顶点，即 $\frac{(n-1)(n-2)}{2} + 1$

图的存储及基本操作

1. 十字链表与邻接多重表：BV1gqAZeyEaB
2. 设图 G 的邻接矩阵为 A ， A^n 的元素 a_{ij}^n 等于从顶点 i 到 j 的长度为 n 的路径的数目
 - 求 A^n
 - 对于无向图，是对称阵
 - 特别地，求 A^2 时，主对角线上的元素为对应节点的度
 - 数长度为 n 的路径条数即可

图的应用

最小生成树

1. 最小生成树是生成子图，具有树的性质，即边数为顶点数 - 1
2. 最小生成树不保证任意两个顶点之间的路径是最短路径
3. 唯一性总结
 - **MST 唯一的充要条件**：图中所有边的权重互不相同，且对于任意不在 MST 中的边 (u, v) ，其权重严格大于 MST 中 u 到 v 路径上所有边的权重
 - 边数 = $n - 1$ 时 MST 一定唯一；边数 $> n - 1$ 时 MST 可能唯一也可能不唯一

dijkstra

1. 算法流程

- 初始化 `dis`，若源点 s 到顶点 i 有边，就初始化为边的权值，否则初始化为 ∞
- 初始化顶点集 S 只包含源点 s
- 每次选择一个到顶点 s 距离最近的顶点 u 加入顶点集 S ，并判断由源点 s 绕行顶点 u 后到任一顶点 v 是否距离更短，若距离更短则将 dis_v 更新为 $dis_u + W_{uv}$ ，重复这个过程，直到所有顶点都加入顶点集 S

拓扑排序

- 1. 拓扑序列包含有向无环图的所有顶点

AOV 与 AOE 网对比

特征	AOV 网 (Activity On Vertex)	AOE 网 (Activity On Edge)
顶点含义	表示 活动	表示 事件 (活动的开始/结束点)
边含义	表示活动间的 依赖关系 (无权值)	表示 活动 (带权值, 记录持续时间)
关注目标	活动的 执行顺序 (拓扑排序)	活动的 时间开销 (关键路径计算)
权值位置	无边权	边有权值 (活动耗时)
核心应用	解决任务调度问题	计算工程最短工期、关键活动
典型问题	能否有序完成所有活动?	完成工程的最短时间是多少?
结构示例	顶点: 任务; 边: 任务A→任务B	边: 任务 (权值=耗时); 顶点: 事件

关键路径

- 1. 存在多条关键路径, 当且仅当它们长度相等; 延长任一关键活动必然延长工期; 但缩短任一关键活动不一定缩短工期

第七章 查找

平均查找长度 ASL

- 1. 衡量查找算法效率的最重要的指标 $ASL = \sum_{i=1}^n P_i C_i$
 - P_i : 查找第 i 个数据元素的概率, 一般认为 $P_i = \frac{1}{n}$
 - C_i : 找到第 i 个数据元素所需进行的比较次数

2.	查找算法	流程描述	$ASL_{success}$	ASL_{fail}	时间复杂度	注
	一般线性表的顺序查找	从前往后/从后往前找	$\frac{n+1}{2} \quad (1)$	$n + 1$	$O(n)$	
	有序线性表的顺序查找	从前往后找, 若第 i 个元素小于 $target$, 第 $i + 1$ 个元素大于 $target$, 则直接失败	$\frac{n+1}{2}$	$\frac{n}{2} + \frac{n}{n+1} \quad (2)$	$O(n)$	相比一般线性表的顺序查找降低了 ASL_{fail}
	折半查找 (二分查找)	每次取中, 维护左右边界	近似解: $\frac{n+1}{n} \log_2 (n + 1) - 1 \quad (3)$ 解析解: $h - \frac{2^h - h - 1}{n} \quad (4)$	$(h + 1) - \frac{2^h}{n+1} \quad (5)$ 其中 $h = \lceil \log_2 (n + 1) \rceil$	$O(\log_2 n)$	要求有序表, 且顺序存储
	分块查找	块内无序, 块间有序, 索引表包含各块的最大关键字和各块中第一个元素的地址	$\frac{s^2 + 2s + n}{2s} \quad (6)$		$O(\sqrt{n})$	分块查找是 内存中大型半有序数据集 的高效检索方案, 是 B+ 树等复杂索引的思想基础

- (1): 从后往前: $\sum_{i=1}^n P_i (n - i + 1) = \frac{n+1}{2}$; 从前往后: $\sum_{i=1}^n P_i i = \frac{n+1}{2}$
- (2): 有 $n + 1$ 个失败区间, 令 q_j 为到达第 j 个失败节点的概率, l_j 为到达第 j 个失败节点的比较次数

$$\begin{aligned}
& q_j = \frac{1}{n+1} \\
& l_j = \begin{cases} j & j = 1, 2, \dots, n \\ n & j = n+1 \end{cases} \\
& ASL_{\text{fail}} = \sum_{j=1}^{n+1} q_j l_j = \frac{n(n+3)}{2(n+1)} = \frac{n}{2} + \frac{n}{n+1} \quad \sim \frac{n}{2}
\end{aligned}$$

○ (3)/(4)/(5): 二分查找的平均查找长度推导

- 二分查找的判定树为一棵平衡二叉树

名称	含义
n	序列长度
h	树高, 从 1 开始
内部节点	判定树中的非叶子结点, 代表序列中的元素, 参与比较
外部节点	判定树中的叶子结点, 代表失败区间, 不参与比较
t	第 h 层内部节点数
s	第 h 层节点数
m	外部节点数
L_h	第 h 层外部节点数
L_{h+1}	第 $h+1$ 层外部节点数
S	外部节点深度和

- ASL_{success} 推导:

- 近似解: 假设判定树为完全二叉树, 则节点个数 $n = 2^h - 1$, 即树高 $h = \lceil \log_2(n+1) \rceil$, 于是有

$$ASL_{\text{success}} = \frac{1}{n} \sum_{i=1}^h l_i = \frac{1}{n} \sum_{i=1}^h i \cdot 2^{i-1} = \frac{n+1}{n} \log_2(n+1) - 1 \approx \log_2(n+1) - 1$$

- l_i : 判定树第 i 层所有节点的总比较次数, i 从 1 开始, 对应深度 $i-1$, 该层节点数为 2^{i-1} , 每个节点的比较次数为 i , 故 $l_i = i \cdot 2^{i-1}$
- 使用错位相减法求解和式即可

- 解析解:

$$\begin{aligned}
& \text{在第 } h \text{ 层, 显然 } s = 2^{h-1} \\
& m = L_h + L_{h+1} = (s - t) + 2t = s + t = n + 1 \\
& \text{即 } t = m - s \\
& \text{在第 } 1 \sim h-1 \text{ 层, } l_i = i \cdot 2^{i-1} \\
& \text{在第 } h \text{ 层, } l_h = ht = h(m - s) = h(m - 2^{h-1}) \\
& \text{于是, } ASL_{\text{success}} = \frac{1}{n} \sum_{i=1}^h l_i = \frac{1}{n} \sum_{i=1}^{h-1} i \cdot 2^{i-1} + h(m - 2^{h-1}) \\
& = \frac{1 - 2^h + hm}{n} \quad \text{错位相减}
\end{aligned}$$

$$\begin{aligned} &= \frac{1 - 2^h + h(n + 1)}{n} \\ &= h - \frac{2^h - h - 1}{n} \end{aligned}$$

■ ASL_{fail} 推导:

由 $ASL_{success}$ 推导过程, 有:

$$\begin{aligned} & \\ \begin{aligned} L_h &= s - t = s - (m - s) = 2^{h-1} - (m - 2^{h-1}) = 2^h - m \\ L_{h+1} &= 2t = 2(m - s) = 2(m - 2^{h-1}) \\ \therefore S &= L_h \cdot (h-1) + L_{h+1} \cdot h \\ &= (2^h - m)(h-1) + 2(m - 2^{h-1})h \\ &= m(h+1) - 2^h \\ \therefore ASL_{fail} &= \frac{1}{m} \cdot S = \frac{m(h+1) - 2^h}{m} = (h+1) - \frac{2^h}{n+1} \quad \text{其中 } h = \lceil \log_2(n+1) \rceil \end{aligned} \end{aligned}$$

- o (6): 长度为 n 的查找表均匀地分为 b 块, 每块有 s 个数据元素, 在等概率情况下, 若在块内和索引表均采用顺序查找, 则有

$$ASL = L_I + L_S = \frac{b+1}{2} + \frac{s+1}{2} = \frac{s^2 + 2s + n}{2s}$$

- 若 $s = \sqrt{n}$, $ASL_{min} = \sqrt{n} + 1$

二叉排序树

删除

待删除的节点

1. 没有孩子: 即是叶结点, 直接删除, 结束
2. 只有左子树/只有右子树: 直接代替, 结束
3. 左右子树都有: 直接后继 (或前驱) 代替值, 然后删除直接后继 (或前驱), 转换为前两种情况

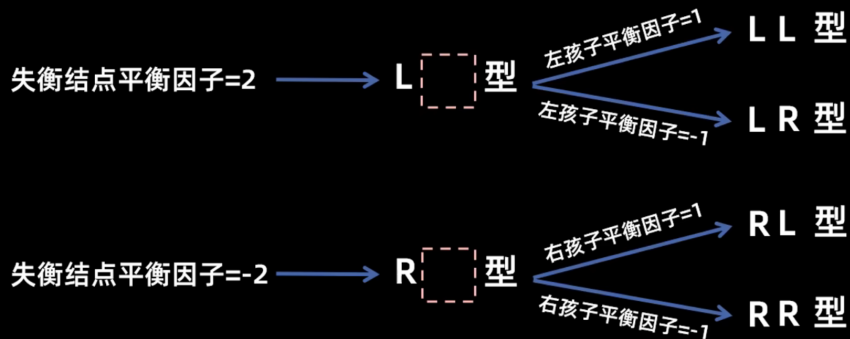
AVL 树

调整

名称	解释	流程 (对于失衡节点)
LL (右旋)	插入位置在失衡节点的 左 孩子的 左 子树	
RR (左旋)	插入位置在失衡节点的 右 孩子的 右 子树	
LR	插入位置在失衡节点的 左 孩子的 右 子树	左旋 左孩子, 然后 右旋
RL	插入位置在失衡节点的 右 孩子的 左 子树	右旋 右孩子, 然后 左旋

四种失衡情况

LL型	RR型	LR型	RL型
失衡结点: 平衡因子 = 2	失衡结点: 平衡因子 = -2	失衡结点: 平衡因子 = 2	失衡结点: 平衡因子 = -2
失衡结点左孩子: 平衡因子 = 1	失衡结点右孩子: 平衡因子 = -1	失衡结点左孩子: 平衡因子 = -1	失衡结点右孩子: 平衡因子 = 1
右旋	左旋	左旋左孩子, 然后右旋	右旋右孩子, 然后左旋



- 插入一个节点后如果导致多个节点失衡，只需调整距离插入节点最近的失衡节点，其他失衡节点会自然平衡
- 删除节点后需要依次对祖先节点检查并调整

红黑树

红黑性质

1. 每个节点要么是红色的，要么是黑色的
2. 根节点是黑色的
3. 叶结点（虚拟的外部节点，即 NIL）是黑色的
4. 不存在两个相邻的红节点（红节点的父节点和子节点均是黑色的）
5. 对每个节点，从该节点到任意一个叶结点的简单路径上，所含黑节点的数量相同

口诀记忆：左根右，根叶黑，不红红，黑路同

与 AVL 树的效率比较

由于黑路同性质，红黑树最长路径一定是黑红相间的，于是 **最长路径不超过最短路径的两倍**，即任一节点左右子树高度相差不超过两倍

其与 AVL 树相比，平衡条件更宽松，故 AVL 树高略优于红黑树，即 **AVL 树查询效率略优于红黑树**

但是 AVL 树在增删时需要消耗更多时间调整，**红黑树的增删效率高于 AVL 树**

插入

插入节点一定初始化为红节点，因为这样只可能破坏 **根叶黑** 和 **不红红** 两条性质

插入节点后，若红黑树性质被破坏，分三种情况调整：

1. 插入节点是根节点：直接变黑，结束
2. 插入节点的叔叔节点是红色：将 **父/叔/爷** 反色，设置爷爷节点为插入节点，继续调整
3. 插入节点的叔叔节点是黑色：以爷爷节点作为旋转点，判断旋转类型（LL/RR/LR/RL），**旋转**，然后对旋转中心点和旋转点反色

删除

指针	含义
d	待删节点 (deletion)
p	d 的父节点 (parent)
s	d 的兄弟节点 (sibling)
r	d 的兄弟节点的子节点

使用二叉排序树删除法，因为第三种情况可化为前两种，只考虑前两种情况。对于待删除节点

1. 没有孩子

- 若为红节点：**直接删除**，不会破坏红黑性质，结束
- 若为黑节点：必然破坏黑路同，将待删节点变为双黑结点（即认为其有双重黑色，暂时抵消删掉的黑色）
 - 若兄弟是黑色
 - 若兄弟至少有一个红孩子：先 **变色**，变色赋值如下，然后 **旋转**，旋转点为 p ，旋转后，**双黑变单黑**，结束
 - 若为 LL/RR 型： $r \leftarrow s; s \leftarrow p; p$ 变黑
 - 若为 LR/RL 型： $r \leftarrow p; p$ 变黑
 - 若兄弟的孩子都是黑色：**兄弟变红，双黑上移**
 - 若双黑上移遇到红节点或根节点，直接变为单黑，结束
 - 若双黑上移遇到黑节点，先变色，变色赋值如下，然后旋转，旋转点为 p ，旋转后，双黑变单黑，结束
 - 若为 LL/RR 型： $r \leftarrow s; s \leftarrow p; p$ 变黑
 - 若为 LR/RL 型： $r \leftarrow p; p$ 变黑
 - 若兄弟是红色：兄父反色，朝双黑旋转，保持双黑继续调整

2. 只有左孩子/只有右孩子：**代替后变黑**，结束

对于红黑树，只可能出现一黑一红，红节点位于黑节点的左/右证：

在一黑一红情况下，只可能出现以下情况

- 1. 添加红节点，违反不红红
- 2. 添加黑节点，违反黑路同

若为一红一黑，黑节点位于红节点的左右，违反黑路同，故红节点不可能只有左子树/右子树故只可能出现一黑一红，红节点位于黑节点的左/右

删除-算法导论中的陈述

1. 删除预处理

情况	操作
待删节点有两个孩子	转化为前驱/后继删除
待删节点是红色叶子	直接删除
待删节点是黑色叶子	设为双黑节点进入调整流程

情况	操作
待删节点有单个孩子	用孩子替代并染黑

2. 双黑调整流程（循环直到满足稳态）

B 树

性质

- 平衡**：所有叶结点在同一层（内部节点的最后一层）
- 有序**：节点内有序（任一元素的左子树都小于它，右子树都大于它）
- 多路**：对于 m 阶 B 树， $n = b - 1$
 - 根节点中关键字个数 $1 \leq n_r \leq m - 1$
 - 根节点分支个数 $2 \leq n_b \leq m$
 - 普通节点中关键字个数 $\lceil \frac{m}{2} \rceil - 1 \leq n \leq m - 1$
 - 普通节点分支个数 $\lceil \frac{m}{2} \rceil \leq b \leq m$

对于 **m 阶 B 树** ($m \geq 3$)，树高 h ，定义根节点高度为 1，最小度数 $t = \lceil m/2 \rceil$ ，有：

参数	最小值	最大值	说明
根节点关键字数	1	$m - 1$	根节点至少 1 个关键字（除非树为空）
根节点孩子数	0 (若 $h = 1$) 或 2 (若 $h \geq 2$)	m	$h = 1$ 时无孩子； $h \geq 2$ 时至少 2 个孩子
非根节点关键字数	$t - 1$	$m - 1$	$t = \lceil m/2 \rceil$ ；适用于所有非根节点（含叶子节点）
非根节点孩子数	0 (叶子节点) 或 t (内部节点)	m	叶子节点无孩子；内部节点至少 t 个孩子
关键字总数 n	$2t^{h-1} - 1$	$m^h - 1$	关键字总数的全局范围
树高 h	-	-	给定参数（根高度为 1），不涉及范围
节点总数 N	$1 + 2 \frac{t^{h-1} - 1}{t - 1}$	$\frac{m^h - 1}{m - 1}$	节点总数的全局范围

推导：

- $h = 1$ 时，节点个数至少为 1，关键字个数至少为 1
- $h = 2$ 时，节点个数至少为 2，关键字个数至少为 $2(t - 1)$
- $h = 3$ 时，节点个数至少为 $2t$ ，关键字个数至少为 $2t(t - 1)$
- $h = 4$ 时，节点个数至少为 $2t^2$ ，关键字个数至少为 $2t^2(t - 1)$
- 树高为 h 时，节点个数至少为 $2t^{h-2}$ ，关键字个数至少为 $2t^{h-2}(t - 1)$

使用等比数列求和公式求和并化简即可

B 树的高度（磁盘存取次数）

对于 m 阶 B 树共 n 个关键字，有树高 h （定义根节点高度为 1）满足

$$\lceil \log_m (n + 1) \rceil \leq h \leq \log_t (\frac{n + 1}{2}) + 1$$

其中 B 树的最小度数 $t = \lceil \frac{m}{2} \rceil$

插入

- 1. 先找到插入的位置进行插入（插入位置一定在叶结点）
- 2. 如果没有上溢出，无需调整
- 3. 否则中间元素（ $\lceil \frac{m}{2} \rceil$ ）上移，两边分裂（直到没有上溢出为止。若根节点上溢出，则树高加一层）

删除

- 1. 删除非叶节点元素最终都转换成删除叶结点元素
- 2. 删除叶节点元素
 - 1. 没有下溢出无需调整
 - 2. 下溢出
 - 1. 兄弟够借：借（父下来，兄上去）
 - 2. 兄弟不够借：合并（父下移到左，然后右并过来。可能导致父节点下溢出）

B+树

B 树	B+树
B 树所有节点的关键字都有指向对应记录的指针	B+树叶结点包含全部关键字及指向相应记录的指针，非叶节点只作索引
B 树中 m 个分支有 $m - 1$ 个关键字	B+树中 m 个分支有 m 个关键字
B 树顺序查找或范围查找只能中序遍历，效率低	B+树兼顾顺序查找和随机查找，还可方便地进行范围查找

杂项

- 1. 有序表中，二分查找不一定比顺序查找快，因为顺序查找可能第一次比较就找到了
- 2. 二分查找关键字比较序列构成一棵平衡二叉树
- 3. n 个不同节点构造的 **二叉查找树** 形态数
 - 若中序序列确定，则先序序列可能的排列为 n 的 **卡特兰数** $Catalan(n) = \frac{1}{n+1}C_{2n}^n$

推导：已知卡特兰数递推公式 $C_{n+1} = \sum_{i=0}^n C_i C_{n-i}$ 且 $C_0 = 1$

令 $T(n)$ 为 n 个不同节点构成的 BST 形态数，BST 的中序序列为 $\{a_1, a_2, \dots, a_n\}$

设根节点为 a_k ，则

根节点的左子树范围为 $a_1 \sim a_{k-1}$ ，共有 $k - 1$ 个节点，形态数为 $T(k - 1)$

根节点的右子树范围为 $a_{k+1} \sim a_n$ ，共有 $n - k$ 个节点，形态数为 $T(n - k)$

于是有

$$T(n) = \sum_{k=1}^n T(k - 1)T(n - k)$$

令 $i = k - 1$, 则

\$\$

$T(n) = \sum_{i=0}^{n-1} T(i)T(n-1-i) = C_n$

\$\$

4. 高度为 h 的 **平衡二叉树** 形态数: $T(h) = T(h-1)^2 + 2T(h-1)T(h-2)$

推导: 设空树的高度 $h = 0$

令 $T(h)$ 为高度为 h 的平衡二叉树形态数, 显然 $T(0) = 1, T(1) = 1$

对于高度为 h 的平衡二叉树, 根节点的左右子树高度有 3 种情况

1. 左子树高度为 $h-1$, 右子树高度为 $h-1$
2. 左子树高度为 $h-1$, 右子树高度为 $h-2$
3. 左子树高度为 $h-2$, 右子树高度为 $h-1$

于是

$T(h) = T(h-1)T(h-1) + T(h-1)T(h-2) + T(h-2)T(h-1) = T(h-1)^2 + 2T(h-1)T(h-2)$

5. 平衡二叉树满足平衡的 **最少** 节点数递推公式 $n_0 = 0, n_1 = 1, n_2 = 2, n_h = 1 + n_{h-1} + n_{h-2}$

平衡二叉树满足平衡的 **最多** 节点数公式 $n = 2^h - 1$

散列表

ASL

- 散列表长度为 n , 散列函数 $H(k) = k \% m$, 插入 t 个元素
 - $ASL_{success}$: 在构建散列表时表明每个元素的查找长度 a_i , $ASL_{success} = \frac{1}{t} \sum_{i=1}^t a_i$
 - ASL_{fail} : 需要分析散列表的每个位置, 注意取模
 - 若没有元素, 查找长度为 1
 - 若有元素, 需要判断该元素到最近的空节点的距离 (从空节点 1 开始数)
 - 若该元素已被删除 (del), 也需要判断该元素到最近的空节点的距离

第八章 排序

简单选择排序

1. 简单选择排序的比较次数与序列初始状态无关, 始终为 $\frac{n(n-1)}{2}$

堆排序

1. 小根堆中关键字最大的元素的存储范围为 $\lfloor n/2 \rfloor + 1 \sim n$, 因为二叉树的最后一个非叶节点存储在 $\lfloor n/2 \rfloor$
2. 求比较次数, 每次与左右儿子都要比较, 要考虑最后一次比较 (不交换)
3. 需要注意区分题目求的是比较次数还是交换次数

归并排序

1. 一般而言, 对于 N 个元素进行 k 路归并排序时, 排序的趟数 m 满足 $k^m = N$, 从而 $m = \log_k N$, 又考虑到 m 为整数, 因此 $m = \lceil \log_k N \rceil$, 显然与序列初始状态无关
2. 归并两个长度分别为 m 和 n 有序表
 - 当一个表的最小元素比另一个表的最大元素还大时, 比较的次数是最少的, 仅比较 $\min(m, n)$ 次
 - 当两个表中的元素依次间隔地比较时, 比较的次数最多, 为 $m + n - 1$ 次

外部排序

1. 核心问题：当数据量太大（超过内存容量）时，无法一次性加载到内存排序。**核心目标是最小化磁盘 I/O 次数**（读写数据最耗时）
2. 采用归并排序，对于 r 个初始归并段进行 k 路归并排序时，树高 h 和归并趟数 S 满足 $h - 1 = \lceil \log_k r \rceil = S$
3. 要减少 S ，显然可以增大 k 和减少 r ，即增加归并路数和减少初始归并段数

生成初始归并段——置换-选择排序 (Replacement Selection)

减少初始归并段数，生成更长的初始有序段，减少归并趟数（从 WA 中选择 $MINIMAX$ 可使用败者树）

1. 内存开辟工作区 WA ，容量为 W ，从原始待排文件 FI 中输入 w 个记录到 WA 中
2. 设置 $MINIMAX$ 为 WA 中关键字最小的记录
3. 每轮将 $MINIMAX$ 记录输出到初始归并段输出文件 FO 中去，并输入下一个记录到 WA ，设置 $MINIMAX$ 为比原 $MINIMAX$ 大的 WA 中最小的记录
4. 直到从 WA 中选不出新的 $MINIMAX$ 为止，此时 FO 中就是一个初始归并段（可输出一个归并段的结束标志到 FO ）
5. 重复 2 ~ 3，直到 WA 为空

归并阶段——败者树 (Loser Tree)

增加归并路数，若使用多路平衡归并（ k 路归并），但是 k 增大时，内部比较次数剧增，需要使用败者树优化

- **完全二叉树**：叶子节点存数据，内部节点记录 "败者"
- 根节点记录最终胜者（最小关键字的段号）
- **调整复杂度**： $O(\log K)$ （远优于普通比较的 $O(K)$ ）

优化归并顺序——最佳归并树 (Optimal Merge Tree)

若初始归并段长度不相等时，使用最佳归并树组织初始归并段的归并顺序

- 利用哈夫曼树贪心的思想自下而上优先归并长度较小的归并段，归并段的长度即为哈夫曼树的权重
- 最佳归并树是一颗 **严格 k 叉树**，每个节点的度只能为 0 或 k
- 若初始归并段不足以构成严格 k 叉树，需添加长度为 0 的 **虚段**

1. 求虚段个数，对于 n 个初始归并段进行 k 路归并，因为每次归并会将 k 个节点合并为一个，净减少 $k - 1$ 个节点，总体来看是将 n 个节点归并为一个，净减少 $n - 1$ 个节点，于是有 $(k - 1) | (n - 1)$ ，即 $(n - 1) \% (k - 1) = 0$ 。所谓添加虚段，就是在不整除时增加节点个数使之整除，于是虚段个数 d 满足 $(n + d - 1) \% (k - 1) = 0$ ，取最小值即可
2. 此外，最佳归并树可用于求解严格 k 叉树的带权路径长度 WPL 最小值

外部排序流程

杂项

1. k 路归并需要 k 个输入缓冲区和 1 个输出缓冲区，若要支持并行需要 $2k$ 个输入缓冲区和 2 个输出缓冲区，每个缓冲区对应一个文件

附录：杂项

排序类别	排序算法	时间复杂度			辅助空间复杂度	稳定性
		最好	最坏	平均		
插入排序	直接插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
	折半插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
	希尔排序	$O(n)$	$O(n^2)$	$O(n^{1.3})$	$O(1)$	不稳定
交换排序	冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
	快速排序	$O(n\log_2 n)$	$O(n^2)$	$O(n\log_2 n)$	$O(\log_2 n)$	不稳定
选择排序	简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
	堆排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(1)$	不稳定
归并排序		$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n)$	稳定
基数排序		$O(d(n+r))$	$O(d(n+r))$	$O(d(n+r))$	$O(r)$	稳定

排序算法	趟数与初始状态的关系	固定趟数 (若无关)	可变范围 (若有关)	关键原因
选择排序	无关	$n - 1$	-	每趟选择最小元素，固定循环。
插入排序	无关	$n - 1$	-	外部循环固定，趟数不变。
归并排序	无关	$\lceil \log_2 n \rceil$	-	分治合并层数固定。
堆排序	无关	$n - 1$ (提取阶段)	-	构建堆后提取次数固定。
希尔排序	无关	增量序列长度 (如 $O(\log n)$)	-	增量序列预定义，趟数固定。
基数排序	无关	数字位数 d	-	每趟处理一位，固定趟数。
计数排序	无关	2 趟 (常数)	-	计数和输出趟数固定。
桶排序	无关	2 趟 (分配和收集)	-	主要框架趟数固定。
冒泡排序	有关	-	1 到 $n - 1$	提前终止机制使趟数可变。
快速排序	有关	-	$O(\log n)$ 到 $O(n)$	分区不平衡导致递归深度可变。

大题总结

算法题

图的应用题

外部排序题

哈夫曼树题

散列表题